DOCENTE	Paolo BELLONI [paolo.belloni@its-ictpiemonte.it]
Unità Formativa	**Elementi di Virtualizzazione**
Argomento	La virtualizzazione della memoria

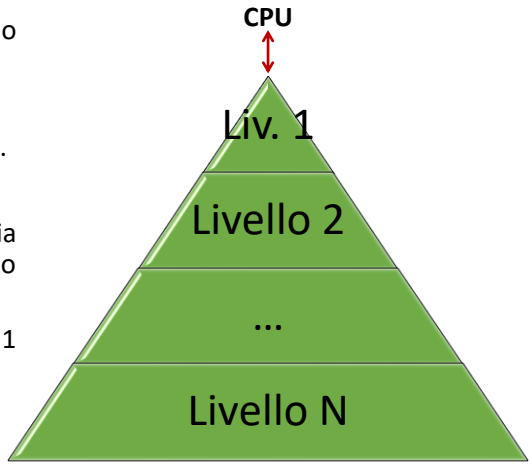
Richiami sulla memoria 1/4

- La memoria principale può agire come “cache” per la memoria di massa (dischi, nastri).
- Si parla in questo caso di memoria virtuale:
  - I programmi sono compilati rispetto ad uno spazio di indirizzamento virtuale, diverso da quello fisico;
  - Il sistema di memoria virtuale traduce gli indirizzi virtuali in indirizzi fisici.
- Vantaggi:
  - illusione di avere una maggiore memoria fisica;
  - meccanismi di protezione (gli spazi virtuali di programmi diversi mappati su spazi fisici differrenti).

Richiami sulla memoria 2/4

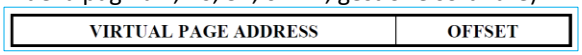
- La memoria e organizzata in modo gerarchico (a livelli):
- Liv. 1: memoria più veloce(costosa).
- Liv. N: memoria più lenta (economica).

- Il processore indirizza tutta la memoria di Liv. N, ma accede direttamente solo al Liv. 1.
- La pagine cercate si trovano al livello 1 con maggiore probabilità.



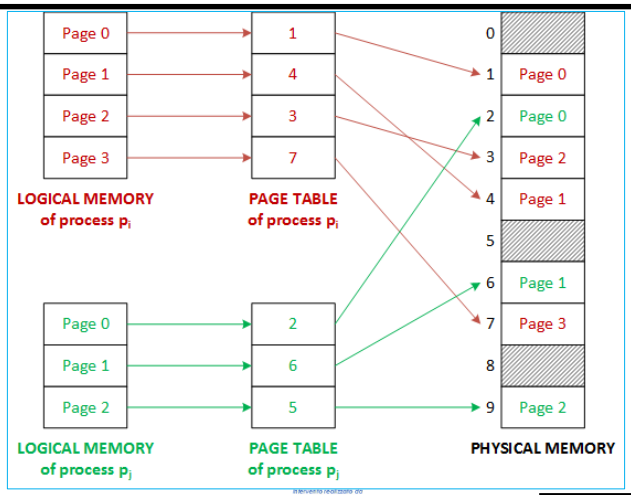
Richiami sulla memoria 3/4

- Ad ogni istante solo una parte delle pagine di memoria “virtuale” è presente nella memoria fisica.
- Quando la pagina cercata non è in memoria e deve essere letta da disco, si parla di page fault (miss).
- Il costo dei page fault è estremamente elevato (milioni di cicli), e questo influenza molte scelte (dimensioni della pagina 4, 16, 32, 64 KB, gestione software).



- L’indirizzo virtuale si suddivide in Virtual Page Address + offset.
- Il Virtual Page Address indirizza le entry di una tabella, detta Page Table, che contiene informazioni sulla presenza / validità delle pagine in memoria.
- In un sistema di memoria virtuale, ogni accesso alla memoria richiede due accessi: prima alla Page Table e quindi alla memoria fisica.
- Per ridurre questo overhead si sfrutta un meccanismo detto TLB (Translation Lookaside Buffer), una sorta di cache della Page Table. La TLB contiene copia delle entry della Page Table riferite recentemente, e, come la cache, può sfruttare l’associatività per ridurre i conflitti.

Richiami sulla memoria 4/4



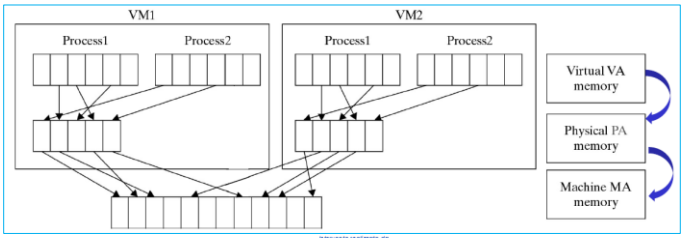
La virtualizzazione della memoria 1/2

- In un ambiente non-virtualizzato:
  - Utilizzando le page table, si passa dalla memoria virtuale alla memoria fisica.
  - Si parla di one-level mapping.
- In un ambiente virtualizzato:
  - Tutte le VM condividono la stessa memoria fisica e hypervisor deve ripartirla tra le VM.
  - Si parla di two-level mapping: dalla memoria virtuale guest alla memoria fisica guest alla memoria fisica host.
- Terminologia:
  - **Guest virtual memory**: memoria visibile alle applicazioni; spazio di indirizzamento virtuale continuo presentato dal guest OS alle applicazioni.
  - **Guest physical memory**: memoria visibile al guest OS.
  - **Host physical memory**: memoria hw reale visibile ad hypervisor.

La virtualizzazione della memoria 2/2

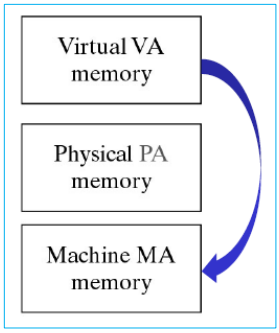
- Two-level memory mapping:
  - Per passare dalla guest virtual memory alla host physical memory serve un mapping a due livelli:

    Guest VA (virtual address) → guest PA (physical address) → host MA (machine address)
  - Guest physical address ≠ host machine address.



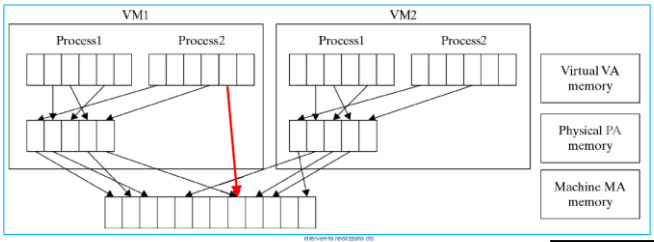
Shadow Page-Table SPT 1/2

- Per evitare la riduzione delle prestazioni derivanti da extra memory mapping, l'hypervisor utilizza delle shadow page-table (SPT) al fine di accelerare il mapping.
  - Mapping diretto guest virtual su host physical memory address.
- SPT mappa direttamente l'indirizzo della guest virtual memory nell'indirizzo della host physical memory.
  - Il guest OS conserva la sua PT per memoria virtuale nello spazio di guest physical memory assegnato.
  - Per ogni dato in guest physical memory, l'hypervisor dovrebbe mapparne l'indirizzo nel dato corrispondente di host physical memory.
  - SPT mappa direttamente l'indirizzo della guest virtual memory nell'indirizzo della host physical memory.
  - L'hypervisor deve mantenere aggiornata la SPT ad ogni cambio che il guest OS attua sulle sue PT.



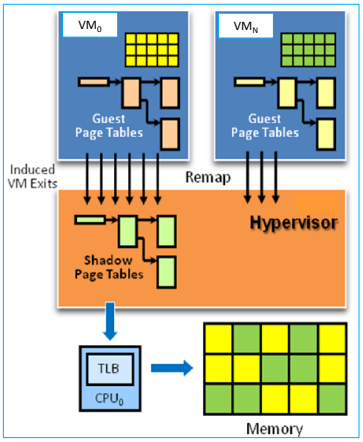
Shadow Page-Table SPT 2/2

- L'hypervisor utilizza l'hw di Translation Lookaside Buffer (TLB) per eseguire il mapping diretto della memoria virtuale nella memoria di macchina fisica, evitando il two-level mapping.
- Quando il guest OS cambia il mapping di guest virtual memory su guest physical memory, l'hypervisor aggiorna le sue SPT per mantenere un mapping diretto aggiornato.



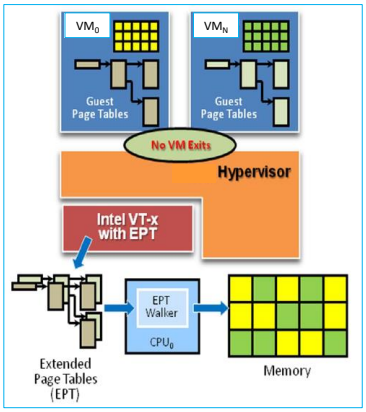
Virtualizzazione della memoria con SPT: Impatti

- Traduzione degli indirizzi.
  - Il guest OS si aspetta indirizzi contigui, mentre la memoria fisica sottostante può apparire discontinua: hypervisor deve preservare questa illusione di continuità.
- Page table shadowing.
  - L'implementazione di SPT è complessa.
  - L'hypervisor intercetta le operazioni di paging e costruisce copia delle PT.
- Overhead.
  - Ogni uscita dalla VM incrementa il tempo di esecuzione.
  - SPT consuma una parte significativa della memoria di host.
  - SPT deve essere sempre sincronizzato con le PT di guest.



Virtualizzazione della memoria con supporto HW

- SPT è una soluzione basata su sw. Esistono tuttavia soluzioni più efficienti, supportate da funzionalità specifiche hw.
  - Second Level Address Translation (SLAT) è una soluzione hardware-assisted per la virtualizzazione della memoria, che traduce guest virtual address in host physical address.
  - Dal 2006, Intel e AMD hanno introdotto nel loro hw il supporto della virtualizzazione di x86 (tecnologia Intel-VTx* e AMD-V**, rispettivamente) includendo la funzione di SLAT con il nome di Extended Page Table (EPT) e Rapid Virtualization Indexing (RVI), rispettivamente.
  - Con SLAT le prestazioni aumentano in modo significativo: misurato un guadagno del 42% rispetto al caso di SPT.



* Intel-VTx = Intel Virtualization Technology, precedentemente denominato Vanderpool (Pentium 4 mod. 662 e 672)

** AMD-V = AMD Virtualization.

The shadow page table code implementation can be critical from an overall performance point of view. To mitigate some of the page table shadowing related overheads, the concept of Extended page tables (EPT) was introduced in Intel Architecture based processors with Intel Virtualization technology enabled. EPT is optionally enabled in Xen when the guest is launched, when EPT is active the EPT based pointer loaded from the VMCS (VM Control Structure) data structure (part of Intel VT-x technology) points to extended page tables. This mechanism allows the Guest OSs full control of the Intel 64/IA-32 page tables which map guest-linear to guest-physical addresses. The new EPT page tables under VMM control and referenced by the EPT base pointer map guest-physical to host-physical addresses to access the memory allocated to the guest domains. This HW based mechanism prevents VM exits due to page faults, INVLPG (invalidate TLB entries), or CR3 accesses resulting in reduced paging related overheads.

CR3 = Control Register 3, used when virtual addressing is enabled, hence when the Paging PG bit is set in CR0. CR3 enables the processor to translate linear addresses into physical addresses by locating the page directory and page tables for the current task.

Hardware-Assisted Memory Virtualization
Some CPUs, such as AMD SVM-V and the Intel Xeon 5500 series, provide hardware support for memory virtualization by using two layers of page tables.

The first layer of page tables stores guest virtual-to-physical translations, while the second layer of page tables stores guest physical-to-machine translation. The TLB (translation look-aside buffer) is a cache of translations maintained by the processor's memory management unit (MMU) hardware. A TLB miss is a miss in this cache and the hardware needs to go to memory (possibly many times) to find the required translation. For a TLB miss to a certain guest virtual address, the hardware looks at both page tables to translate guest virtual address to machine address. The first layer of page tables is maintained by the guest operating system. The VMM only maintains the second layer of page tables.

Performance Considerations
When you use hardware assistance, you eliminate the overhead for software memory virtualization. In particular, hardware assistance eliminates the overhead required to keep shadow page tables in synchronization with guest page tables. However, the TLB miss latency when using hardware assistance is significantly higher. By default the hypervisor uses large pages in hardware assisted modes to reduce the cost of TLB misses. As a result, whether or not a workload benefits by using hardware assistance primarily depends on the overhead the memory virtualization causes when using software memory virtualization. If a workload involves a small amount of page table activity (such as process creation, mapping the memory, or context switches), software virtualization does not cause significant overhead. Conversely, workloads with a large amount of page table activity are likely to benefit from hardware assistance.

By default the hypervisor uses large pages in hardware assisted modes to reduce the cost of TLB misses. The best performance is achieved by using large pages in both guest virtual to guest physical and guest physical to machine address translations.

The option LPage.LPageAlwaysTryForNPT can change the policy for using large pages in guest physical to machine address translations.

L'ottimizzazione delle risorse di memoria

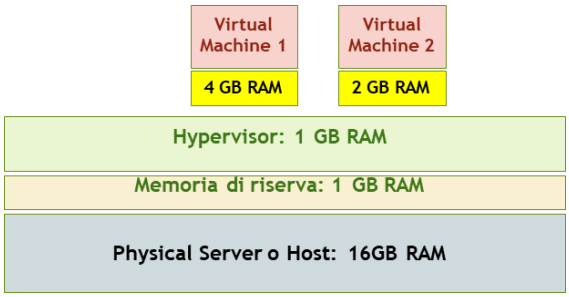
- L'hypervisor gestisce la memoria fisica di una macchina.
  - Parte di questa memoria fisica è occupata direttamente dall'hypervisor.
  - La restante parte di memoria resta disponibile per le varie VM.
- Le VM possono essere configurate per avere più memoria di quanta sia fisicamente disponibile, con un meccanismo chiamato "memory over-commitment".
  - ❖ Di fatto, l'ottimizzazione della memoria viene attuato per permettere il memory over-commitment.
- Le tecniche per gestire la memoria sono:
  - ➢ Transparent Page Sharing (TPS);
  - ➢ Memory Ballooning;
  - ➢ Memory Swapping.

Source: EMC², Cloud Infrastructure and Service, Student Guide, 2011

The hypervisor manages the physical memory of compute systems.
A part of memory is consumed by the hypervisor and rest of the memory is available for virtual machines (VMs).
Hypervisor allows configuring more memory to the virtual machines than what is physically available. This is known as overcommitment of memory. Memory overcommitment is allowed because, typically, some virtual machines are lightly loaded, compared to others. Their memory may be used infrequently, and much of the time, their memory remains idle. Memory overcommitment allows the hypervisor to use memory reclamation techniques to take the inactive or unused memory away from the idle virtual machines and give it to other virtual machines that will actively use it.
For example, consider a physical machine with 4GB physical memory running three virtual machines with 2GB VM memory each. Without memory overcommitment, only one virtual machine can be run because the hypervisor cannot reserve the physical memory for more than one virtual machine. This is because a hypervisor will consume some memory space and each virtual machine has a memory overhead.
In order to effectively support memory overcommitment, the hypervisor must provide efficient physical memory reclamation techniques. A hypervisor supports three techniques to reclaim memory:

- **transparent page sharing,**
- **ballooning, and**
- **memory swapping**.

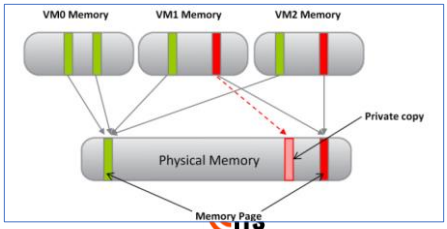
Memory Overhead di Hypervisor



- Hypervisor stesso impone un overhead nel consumo di memoria RAM.
- Overhead: consumo addizionale di risorse del "programma" hypervisor.
- In passato, l'hypervisor poteva occupare una parte significativa della memoria fisica, in alcuni casi fino al 20%.
- La tecnologia hypervisor più recente ha ridotto drasticamente tale quantità.
  - Per ogni VM viene riservata una piccola porzione di memoria addizionale + memoria allocata per l'utilizzo della macchina virtuale.
  - Memoria addizionale utilizzata per funzioni operative (tabelle di mappatura della memoria, etc).
- L'overhead effettivo varia in base all'implementazione dell'hypervisor e alle configurazioni di memoria delle singole macchine virtuali.
- Esempio.
  - Le due VM: 6 GB totali
  - Hypervisor: 1 GB
  - Memoria di riserva: 1 GB. Consente di fronteggiare picchi di carico transitori senza degradare le prestazioni.

Transparent Page Sharing (TPS)

- L'hypervisor identifica le pagine di memoria identiche tra le varie VM (p. es. stesso SO e stessa applicazione su diverse VM) e le mappa sulla stessa pagina fisica, rimuovendo diverse copie della stessa informazione (operazione trasparente alla VM, da cui il nome della tecnica).
  - Utilizza una funzione di hash per confrontare le pagine di memoria e verificarne la similitudine.
  - Quando la pagina fisica è condivisa, diventa read-only.
- Per la scrittura, l'hypervisor tratta le pagine condivise come "copy-on-write".
- Cosa accade in caso di tentativo di scrittura su pagina condivisa da parte della legittima VM?
  - Si genera un fault page non critico (minor fault page);
  - Viene create un copia privata della pagina di memoria dopo la scrittura e su questa viene rimappata la memoria della VM.



Source: EMC², Cloud Infrastructure and Service, Student Guide, 2011

When multiple virtual machines (VMs) are running, some of them may have identical sets of memory content. This presents opportunities to share memory across virtual machines (as well as to share within a single virtual machine). For example, several virtual machines may run the same guest operating system, have the same applications, or contain the same user data. With page sharing, the hypervisor can reclaim the redundant copies and keep only one copy, which is shared by multiple virtual machines in the physical memory. As a result, the total amount of physical memory consumed by virtual machines is reduced and memory overcommitment becomes possible.

A hypervisor identifies redundant page copies by their contents. This means that pages with identical content can be shared regardless of when, where, and how those contents are generated.
The hypervisor scans the content of VM memory for sharing opportunities.
After the candidate virtual machine page content is confirmed to match the content of an existing physical page, the virtual machine memory pointer is updated to point to the shared location, and the redundant physical memory copy is reclaimed. This mapping process is invisible to the virtual machine.
When a write occurs on the shared page, the standard Copy-on-Write (CoW)

technique is used to handle these writes. Any attempt to write to the shared pages will generate a minor page fault.
After a page fault occurs, the hypervisor will transparently create a private copy of the page for that virtual machine. It remaps the pointer to this private copy of the virtual machines. In this way, virtual machines can safely modify the shared pages without disrupting other virtual machines sharing that memory.

Transparent Page Sharing basics

In ESX, a 4KB page backing any guest page with content identical to another 4KB guest page can be shared regardless of when, where, and how those contents are generated. ESX periodically scans the content of guest physical memory for sharing opportunities. For each candidate page, a hash value is computed based on its content. The hash value is then used as a key to look up a global hash table, in which each entry records a hash value and the physical page number of a shared page. If the hash value matches an existing entry, a full bit-by-bit comparison of the page contents between the candidate page and the shared page is performed to exclude a false match. After a successful content match, the guest-physical to host-physical mapping of the candidate page is changed to the shared host-physical page, and the redundant host memory copy is reclaimed.

Transparent Page Sharing with large pages

In hardware-assisted memory virtualization systems, ESX will preferentially back guest physical pages with large host physical pages (2MB contiguous memory region instead of 4KB for regular pages) for better performance. If there is not a sufficient 2MB contiguous memory region in the host (for example, due to memory overcommitment or fragmentation), ESX will still back guest memory using small pages (4KB). ESX will not share large physical pages because:

The probability of finding two large pages that are identical is very low.
The overhead of performing a bit-by-bit comparison for a 2MB page is much higher than for a 4KB page.
However, ESX still generates hashes for the 4KB pages within each large page during page scanning.

In the cases where host memory is overcommitted, ESX may have to swap out pages. Since ESX will not swap out large pages, during host swapping, a large page will be broken into small pages. ESX tries to share those small pages using the pre-generated hashes before they are swapped out. The motivation of doing this is that the overhead of breaking a shared page is much smaller than the overhead of swapping in a page if the page is accessed again in the future.

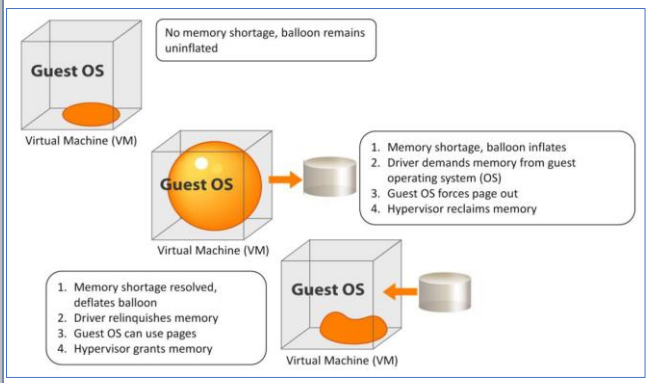
Conclusion

Given the above implementation, one may see zero or very little page sharing in hardware MMU systems when host memory is under committed. Page sharing occurs only when the host starts to swap out pages due to very high memory pressure. Note that with the above features, the ability to reclaim memory through page sharing in ESX is not degraded because ESX defers sharing pages until host free memory is very low.

Memory Ballooning

Vmmemctl.sys driver in Guest OS
sched.mem.maxmemctl paramenter in VM options

- Basata sul principio per cui, se una VM deve rilasciare memoria, la scelta migliore è lasciare che il guest OS identifichi il blocco da liberare, conoscendo quale sia il meno recente.
- Il balloon (palloncino) altro non e' che un driver relativo ad una periferica inesistente (*pseudo-device driver*) installato nel guest OS.
- Il balloon comunica con l'hypervisor, che lo "gonfia" quando la VM deve rilasciare memoria: il balloon richiede un aumento di spazio di memoria al guest OS, che gliele assegna rilasciando pagine di memoria tipicamente meno recenti, in base a propri algoritmi interni usuali.
- L'hypervisor assegna questa memoria rilasciata ad un'altra VM, che ne ha bisogno.
- Quando non c'è esigenza di liberare memoria, il palloncino è sgonfio.
- Il guest OS non è al corrente della comunicazione tra balloon e hypervisor e non conosce lo scopo del pseudo-driver.



Source: EMC², Cloud Infrastructure and Service, Student Guide, 2011

Ballooning is a completely different memory reclamation technique, compared to page sharing.

When a virtual machine (VM) must yield memory, the best thing is to let the guest operating system of the VM select the memory pages to give up. The virtual machine knows which pages have been least recently used and can be freed up. Ballooning technique makes the guest operating system free some of the virtual machine memory.
The balloon driver is installed in the guest operating system as a pseudo-device driver and communicates with the hypervisor. The guest operating system is aware of the fact that the balloon driver is installed but not aware of its purpose. The driver's function is to demand memory from the guest operating system and later to relinquish it under the control of the hypervisor. The guest operating system is not aware of the communication taking place between the balloon driver and the hypervisor.
When a compute is not under memory pressure, no virtual machine's balloon is inflated. But when memory becomes scarce, the hypervisor chooses a virtual machine and inflates its balloon. It instructs the balloon driver in the virtual machine to demand memory from the guest operating system. The guest operating system

complies by yielding memory, according to its own algorithms.
The hypervisor can assign the relinquished pages to other virtual machines that require more memory.
The hypervisor can safely reclaim this physical memory because neither the balloon driver nor the guest operating system relies on the contents of these pages. This means that no processes in the virtual machine will intentionally access those pages to read or write. If any of these pages are reaccessed by the virtual machine for some reason, the hypervisor will treat it as normal virtual machine memory allocation and allocate a new physical page for that virtual machine.

Memory Swapping

- Ogni VM attiva ha bisogno del suo swap file.
  - Lo swap viene utilizzato per liberare memoria RAM: il sistema operativo salva sul disco una porzione della memoria allocata, che quindi può essere liberata e riallocata per i programmi che ne hanno bisogno. Questa porzione contiene i dati che hanno minore probabilità di essere richiesti nel futuro, e in genere sono quelli meno recentemente utilizzati.
  - Lo swap file viene creato quando la VM si attiva e cancellato quando la VM si spegne.
- Lo swap file ha dimensione uguale alla differenza tra il limite di memoria e la memoria riservata alla VM ($SWAP_{vmx} = LIMIT_{vmx} – RESERVATION_{vmx}$).
- L'hypervisor swappa la memoria della VM nel caso in cui la memoria sia scarsa.
- Lo swap di memoria rappresenza l'ultima opzione per gestire l'overcommitment di memoria (dopo page sharing e ballooning), perchè ha un notevole impatto sulle performance.

Source: EMC², Cloud Infrastructure and Service, Student Guide, 2011

As a last effort to manage excessively overcommitted physical memory, the hypervisor will swap the virtual machine's (VM's) memory content to their swap files. When virtual machines are powered on, the hypervisor creates and assigns one swap file to each virtual machine. This swap file stores the virtual machine's memory contents. If a physical machine cannot get enough memory through page sharing and memory ballooning, the hypervisor forcibly reclaims memory
from virtual machines by memory swapping. The hypervisor copies the VM page contents to their corresponding swap files before assigning the pages to the virtual machines that need memory.
The swap file size is determined by the difference between the virtual machine's configured memory (or its memory limit) and its reservation.
Whenever a swap file is actively used, performance is severely penalized. When the virtual machine is powered off, the swap file of that virtual machine is deleted. When the virtual machine is powered back on, the swap file for that virtual machine is re-created.